

UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Escola Superior d'Enginyeries Industrial,
Aeroespacial i Audiovisual de Terrassa

Universitat Politècnica de Catalunya

Escola Superior d'Enginyeries

Industrial, Aeroespacial i Audiovisual de Terrassa

MSc in Aerospace Engineering

Computational Engineering

CSM / 1D finite element method

Alex Carmona Peña

Pau Tusell Tresserras

Autumn 2023

Contents

1 Introduction **1**

2 Weak form **1**

3 Numerical methodology **2**

4 Resolution with linear approximation and comparison with the exact solution **2**
4.1 Displacement results 3
4.2 Stress results 4

5 Resolution with quadratic approximation **5**
5.1 Displacement results 6
5.2 Stress results 7

6 Accuracy and verification **8**

7 Conclusions **9**

8 Annex **11**
8.1 Main code 11
8.2 Jacobian function code 13
8.3 N function code 14
8.4 Exact solution function code 14

1 Introduction

The aim of this assignment is the numerical resolution of a 1-D beam by means of a finite element method regarding some boundary conditions. A representation of the mentioned beam is:

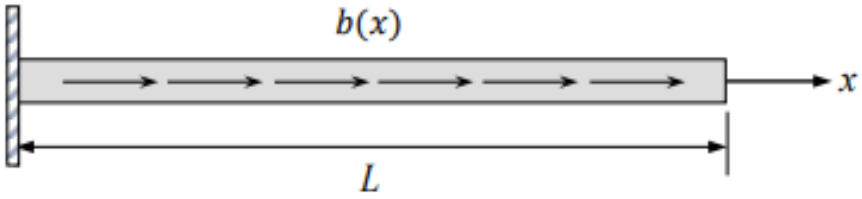


Figure 1: Scheme of the problem to be solved

The first section from the left is attached to a wall with null displacement as the given boundary conditions specify. Plus, the entire beam is under a horizontal force that depends on the location *b(x)*:

$$b(x) = c \cdot exp(\lambda x) \ \ [N/m] \tag{1}$$

$$EA\frac{d^2u}{dx^2} + b(x) = 0 \tag{2}$$

$$u(0) = 0 \tag{3}$$

$$\frac{du}{dx}(L) = 0 \tag{4}$$

Being all the constants known values: A = 120 mm^2, E = 75 GPa, L = 2 m, c = 10000 N/m and $\lambda = 2$.

2 Weak form

The goal of this method is to transform differential equations to an integral form. Hence their numerical resolution is much easier. The procedure starts by adding a test function ($w(x)$) to all the terms, as well as the differential length (dx) to obtain the virtual work. Then equation 2 rewrites as:

$$EA\frac{d^2u}{dx^2}w(x) \cdot dx + b(x)w(x) \cdot dx = 0 \cdot w(x) \cdot dx \tag{5}$$

So it can be integrated.

$$\int_0^L EA\frac{d^2u}{dx^2}w(x) \cdot dx + \int_0^L b(x)w(x) \cdot dx = \int_0^L 0 \cdot w(x) \cdot dx \tag{6}$$

The last term is 0 by mathematical properties, whereas the chain rule is applied to the first one:

$$\int_0^L EA\frac{d^2u}{dx^2}w(x) \cdot dx = EA \cdot w\frac{du}{dx}\Big|_L - EA \cdot w\frac{du}{dx}\Big|_0 - EA\int_0^L \frac{du}{dx}\frac{dw}{dx}dx \tag{7}$$

At this point is very important to note that the test function must satisfy that w(0) = 0. Moreover taking into account the boundary conditions (equations 3 and 4) the term becomes:

$$\int_0^L EA\frac{d^2u}{dx^2}w(x) \cdot dx = EA \cdot w \cdot 0 - EA \cdot 0 \cdot \frac{du}{dx}\Big|_0 - EA\int_0^L \frac{du}{dx}\frac{dw}{dx}dx = -EA\int_0^L \frac{du}{dx}\frac{dw}{dx}dx \tag{8}$$

Now that the first term has been developed, it can be put back to the main equation, which was number 6, leading to the weak form equation:

$$\boxed{-EA\int_0^L \frac{du}{dx}\frac{dw}{dx}dx + \int_0^L b(x)w(x) \cdot dx = 0} \tag{9}$$

3 Numerical methodology

The problem has been solved numerically using the Finite Element Method. The code structure is the following:

1. **Input data:** A, E, L, b(x), w_{kg}, ξ_{kg},...
2. **Compute the exact solution analytically:** will be used to verify the numerical solution later.
3. **Select type of element:** linear or quadratic elements. Depending on the element type, select number of Gauss points n_G and number of nodes per element n_{ne}.
4. **Define matrices:** Nodal coordinates array and nodal connectivity table.
5. **Compute the global stiffness matrix:** To do so, first the Jacobian, B matrices and element stiffness matrices are computed.
6. **Compute the global force vector:** To do so, first the shape functions N are evaluated at each Gauss point, then the coordinates of each Gauss point are computed, and with this the body force at each Gauss point. The element force vector is computed from the body force at the Gauss points, and then the element force vectors are assembled into the global force vector.
7. **Solve the global system of equations:** To do so, first the matrices are partitioned between free and imposed DoFs.
8. **Compute the nodal stresses**
9. **Create displacements and stress vs exact solution plots**
10. **Compute error in the displacements**
11. **Create the error vs element size plot**

4 Resolution with linear approximation and comparison with the exact solution

As a first approach to solving the problem, two-noded linear elements have been employed, making use of linear shape functions for the Gauss integration procedure. The Gauss-Legendre quadrature rule states that the approximation:

$$\int_{-1}^{1} f(\xi) d\xi = \sum_{kg=1}^{ng} w_{kg} f(\xi_{kg}) \tag{10}$$

Is exact for polynomials of degree $2n_G - 1$ or less.

In this case, as a linear shape function is employed, the polynomial is of 1 degree, and thus, 1 Gauss point is required.

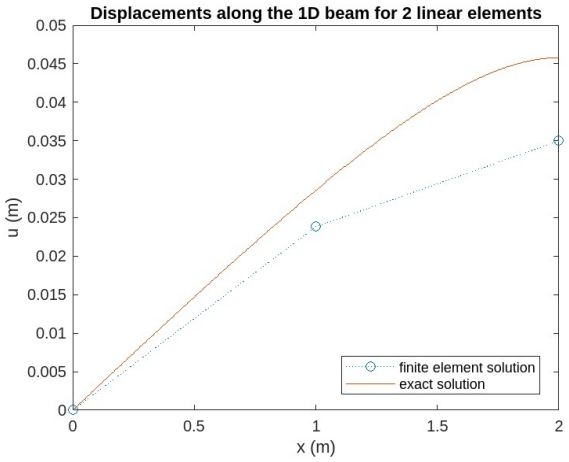
The weight w_{kg} and quadrature point ξ_{kg} take the following values in this case ($n_G = 1$):

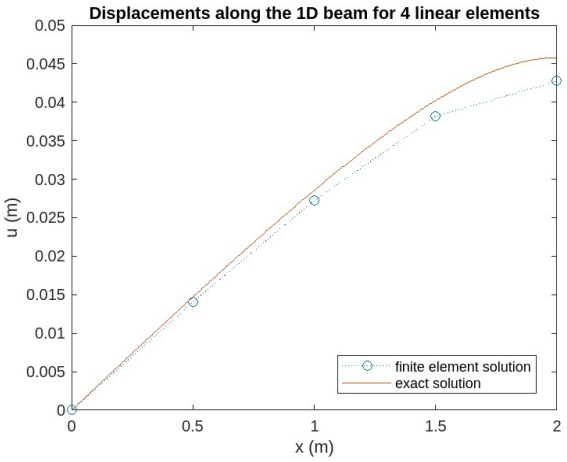
n_G	w_{kg}	ε_k
1	2	0

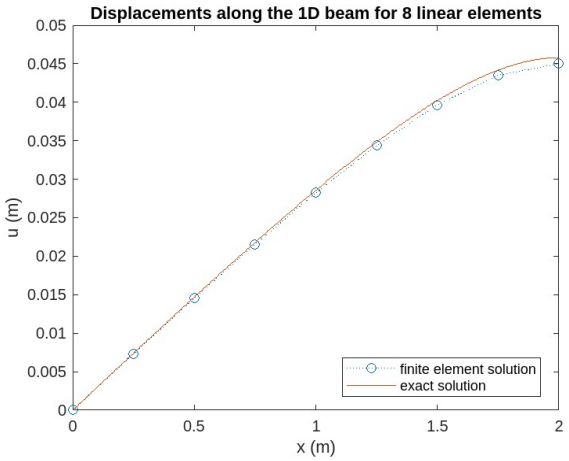
Table 1: Summary of the Gauss quadrature points and weights

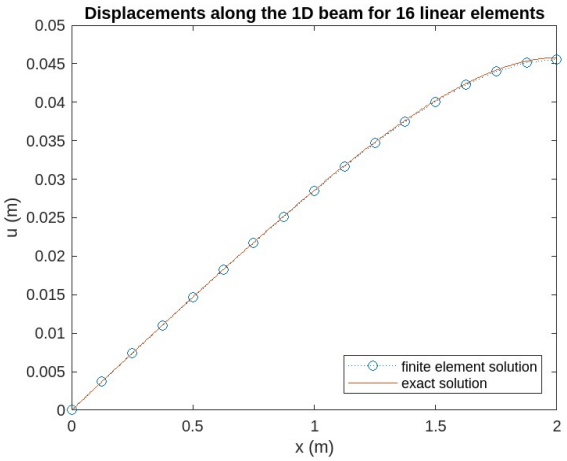
4.1 Displacement results

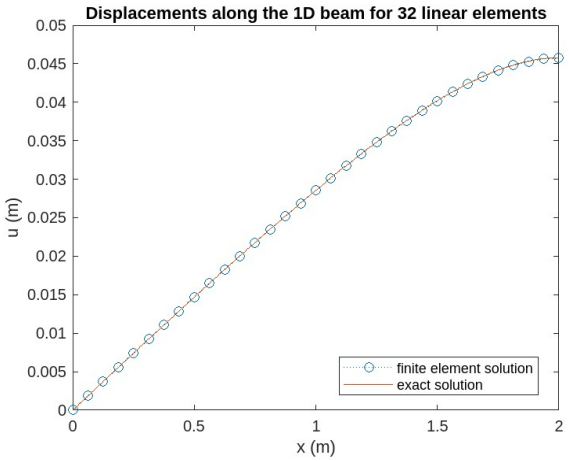
The numerical displacement results, compared with the analytical solution, are shown in the following figures. As can be seen, the solution with few elements is quite poor, however, the displacements are already very well defined with only 8 elements.

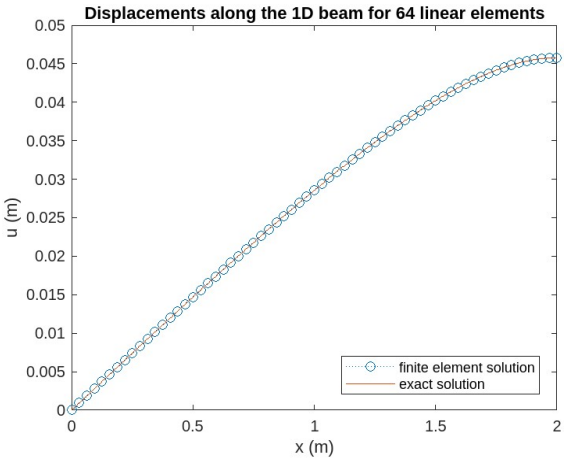


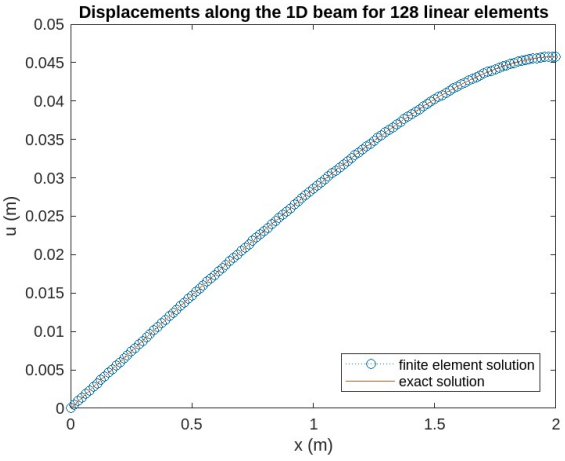






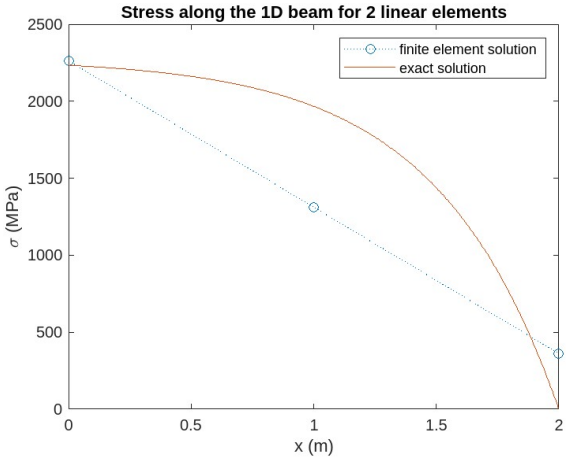


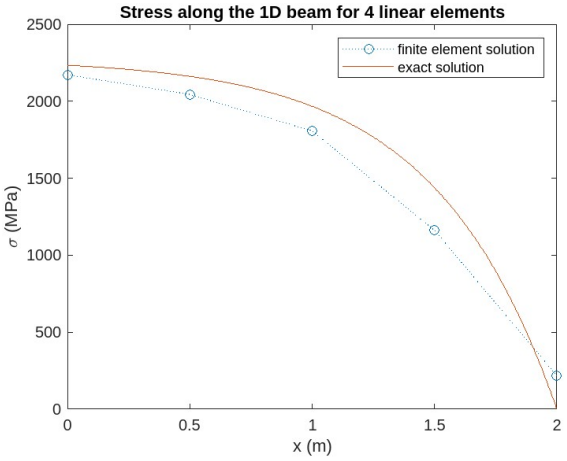


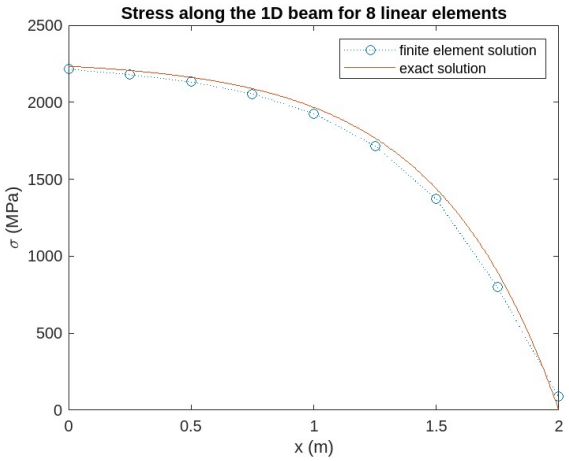


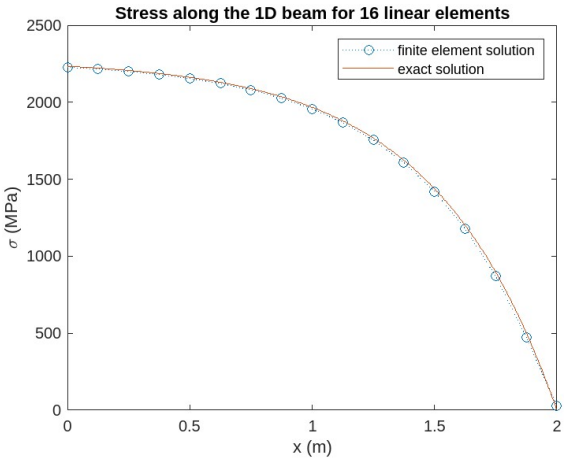
4.2 Stress results

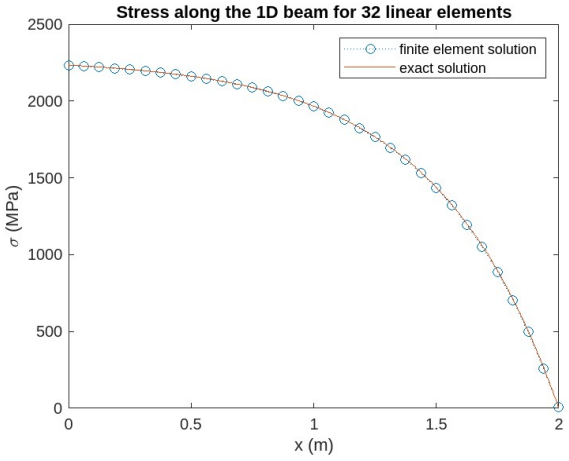
The numerical stress results, compared with the analytical solution, are shown in the following figures. The same case as with the displacements is observed, as the stress is close to being well represented with 8 elements, and is very well represented with 16 elements.

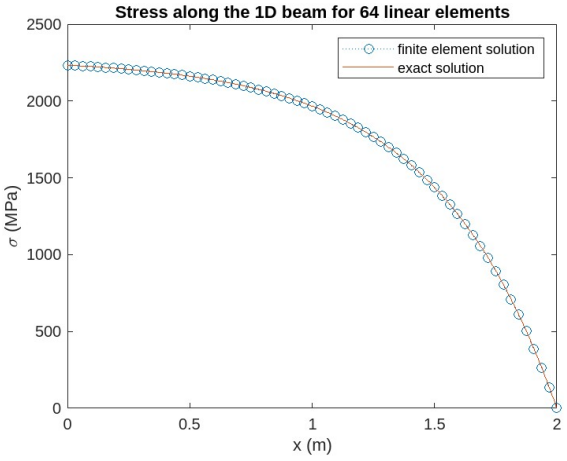


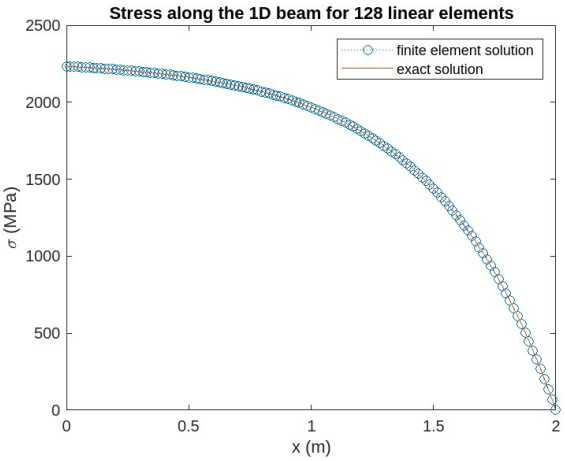












5 Resolution with quadratic approximation

The problem will now be solved with three-noded linear elements using a quadratic shape function. Applying the Gauss-Legendre quadrature rule it can be seen that 2 Gauss points are needed.

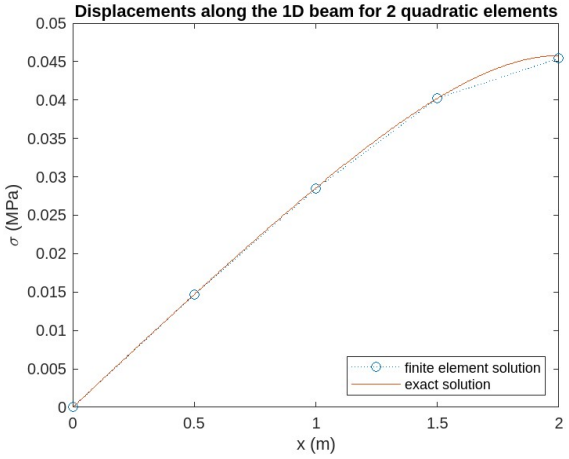
The weights and quadrature points when 2 Gauss points are employed are shown in the following table.

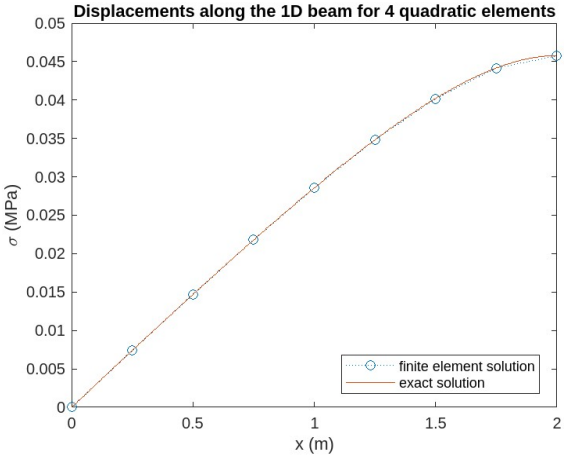
n_G	w_{kg}	ε_k
2	1	$-1/\sqrt{3}$
2	1	$1/\sqrt{3}$

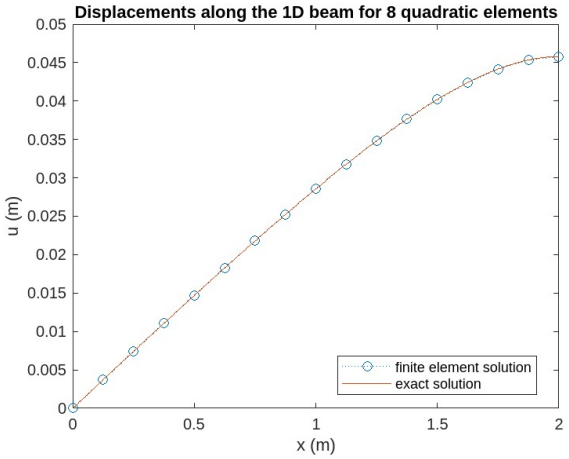
Table 2: Summary of the Gauss quadrature points and weights

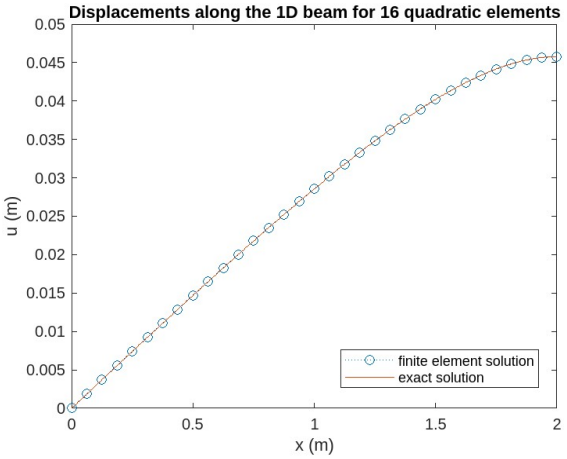
5.1 Displacement results

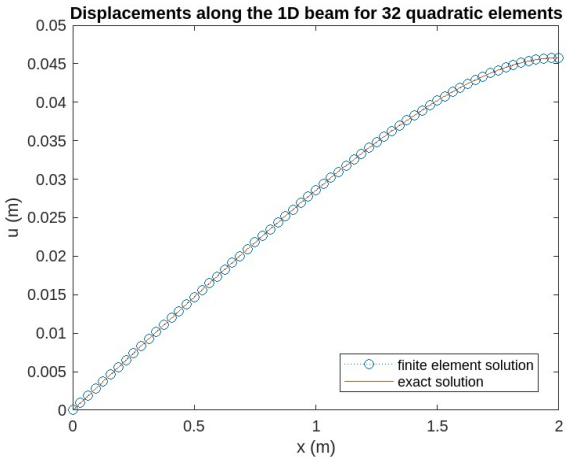
A significant improvement can be seen when using quadratic elements instead of linear elements. With merely 2 elements, the displacements are already very well approximated, while with 4 elements it is practically exact.

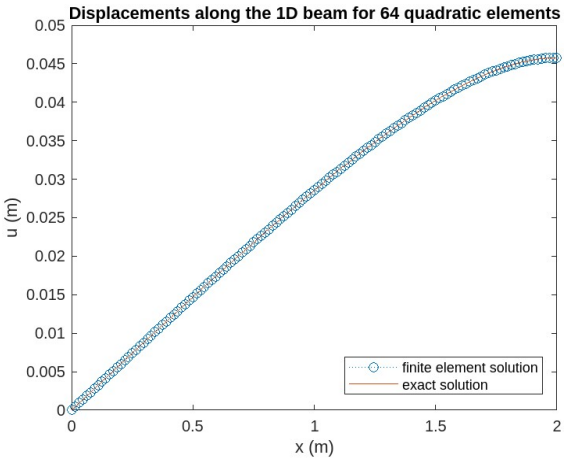


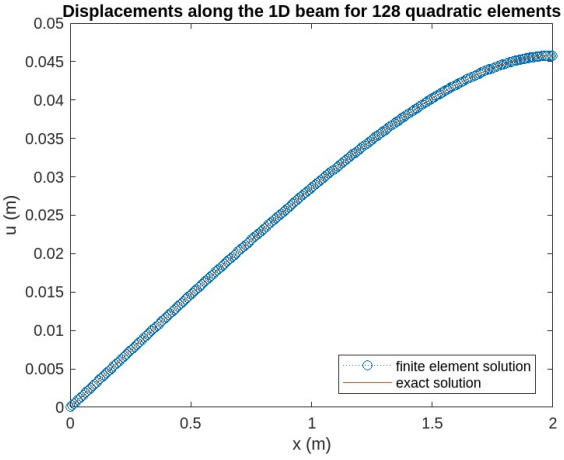






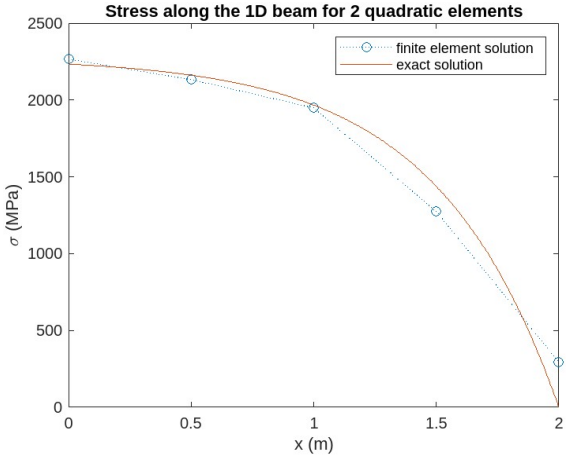


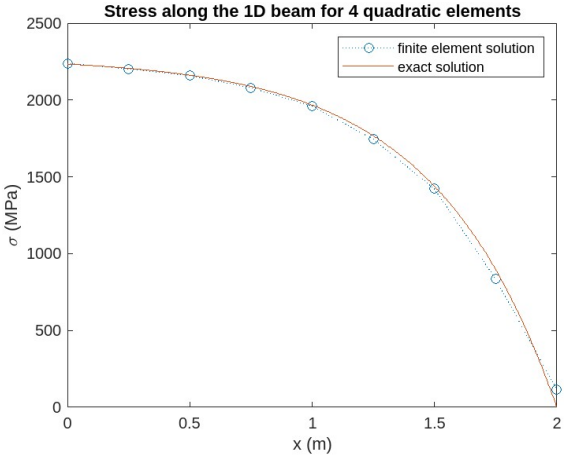


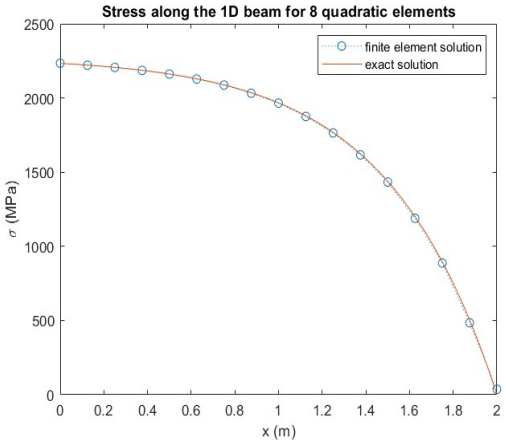


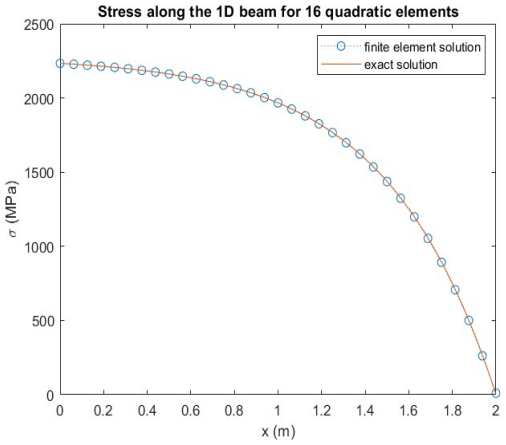
5.2 Stress results

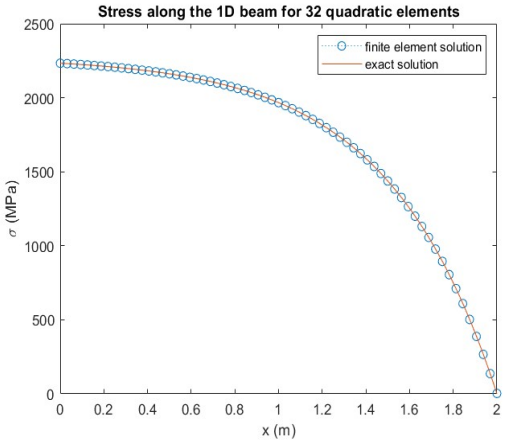
As for the stress with quadratic elements, some deviation is observed with 2 elements, but it is very close to being exact with 4 elements already.

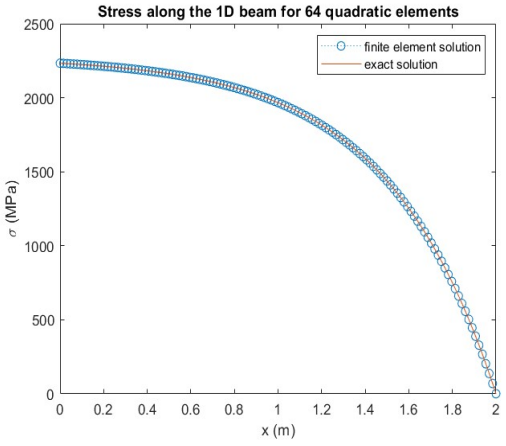


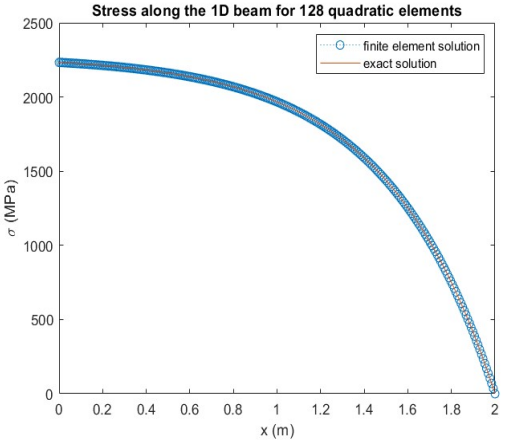












6 Accuracy and verification

The formula to compute the error for every discretization of the beam length is given by:

$$e^u = \frac{||u^{ex}(x) - u^h(x)||_{L_2}}{||u^{ex}(x)||_{L_2}} \tag{11}$$

As this problem is solved with a finite element discretization and the displacements for both exact and approximate solutions have been computed, it's of major interest to rewrite the error equation into a computational form:

$$e^u = \frac{\sqrt{\sum_{i=1}^{nel} ((u^{ex}(x_i) - u^h(x_i))^2)}}{\sqrt{\sum_{i=1}^{nel}((u^{ex}(x_i))^2)}} \tag{12}$$

At this point, the error for both assessed problem types (linear and quadratic) can be easily computed and plotted against the element size.

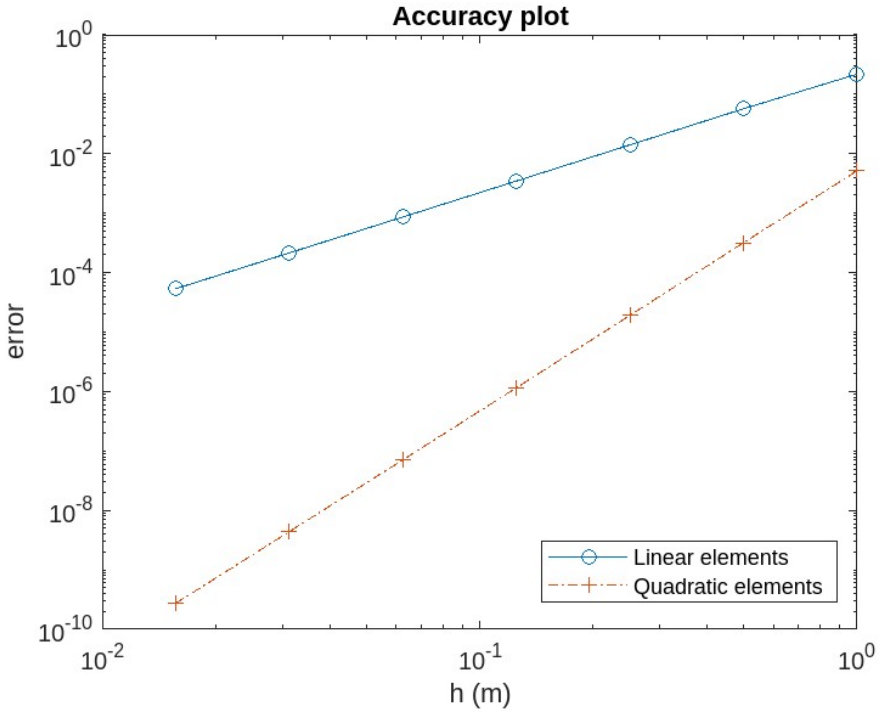


Figure 14: Logarithmic graph of the error versus the beam elements length

Since $h = L/n_{divisions}$, it was expected that for lower h values (meaning more divisions to the beam) one would obtain less error. The low values of error obtained for both cases when increasing the number of divisions verifies the code, as it shows that the solution being obtained is close to the analytical solution.

In addition, for a given element size, the error is significantly lower when quadratic elements are used instead of linear elements, as was already observed in the displacements/stress plots. This shows that the type of element chosen is of major importance regarding the accuracy of the results.

The third observation that can be extracted from the accuracy plot is the slope of each case: The slope is higher with quadratic elements than with linear elements. This gives information about how fast the error decreases when the total number of divisions at the beam is increased.

7 Conclusions

In this assignment, the displacements and stresses along a 1-D cantilever beam subjected to a distributed, location-dependant, axial load have been computed numerically.

The correct solution to the problem has been verified through comparison with the analytical solution.

The problem has been solved with both linear and quadratic elements, employing 1 and 2 Gauss points respectively according to the Gauss-Legendre quadrature rule.

In both cases, solutions close to the exact solution are reached with relatively few elements, especially when employing quadratic elements.

The error in the solution has also been observed to not only be lower when employing quadratic elements than when employing linear elements but has also been observed to decrease more rapidly upon increasing the number of elements when quadratic elements are employed.

8 Annex

8.1 Main code

```
%
clear all
clc


%-------input data---------
% % A = 120; %mm2
% % E = 75; %GPa
% % L = 2; %m
% % c = 1e4; %N/m
A = 120*10^(-6); %m2
E = 75*10^9; %Pa
L = 2; %m
c = 1e4; %N/m
lambda = 2;

n_ne = 2;
%n_G = 2;


w_k = [2 0 0;
       1 1 0;
       5/9 8/9 5/9];

Xi_k = [0 0 0;
       -1/sqrt(3) 1/sqrt(3) 0;
       -sqrt(3/5) 0 sqrt(3/5)];

%exact solutions
[u_exact, du_exact] = compute_exact_solution(A,E,L,c,lambda);
x_vector_exact = linspace(0,L,500);
u_vector_exact = (x_vector_exact*exp(4))/1800 - exp(2*x_vector_exact)/3600 +
    1/3600;
sigma_vector_exact = E*(exp(4)/1800 - exp(2*x_vector_exact)/1800);
sigma_vector_exact=sigma_vector_exact*10^(-6); %conversion to Mpa


%finite element solution
for n_G=1:1:3
    clear u
for cont_n_el = 1:1:7
    n_el = 2^(cont_n_el);
    n = n_ne + (n_el-1)*(n_ne-1);

    h = L/n_el;
    h_vector(cont_n_el) = h;

    x = linspace(0,L,n)';

    Tn = zeros(n_el,n_ne);

    for e=1:1:n_el
        for i = 1:1:n_ne
            Tn(e,i) = (e-1)*(n_ne-1) + i;
        end
    end

    B = zeros(1,n_ne,n_el,n_G);
    Ke = zeros(n_ne,n_ne,n_el);
```

```
    K = zeros(n,n);


    for e=1:1:n_el
        le = x(Tn(e,n_ne))-x(Tn(e,1));
        [Je, dN_dXie] = compute_Jacobian_e(n_G,n_ne,x,e,Tn,Xi_k);
        for k=1:1:n_G
            for i=1:1:n_ne
                B(1,i,e,k)=(1/Je(k))*dN_dXie(i,k);
            end
            Ke(:,:,e)=Ke(:,:,e)+w_k(n_G,k)*Je(k)*E*A*(B(:,:,e,k)'*B(:,:,e,k));
        end
        for i=1:1:n_ne
            for j=1:1:n_ne
                K(Tn(e,i),Tn(e,j)) = K(Tn(e,i),Tn(e,j)) + Ke(i,j,e);
            end
        end
    end


    N = zeros(1,n_ne,n_el,n_G);
    x_G = zeros(n_G,n_el);
    fe = zeros(n_ne,n_el);
    f = zeros(n,1);

    for e=1:1:n_el
        xe = x(Tn(e,:));
        le = x(Tn(e,n_ne))-x(Tn(e,1));
        [Je, dN_dXie] = compute_Jacobian_e(n_G,n_ne,x,e,Tn,Xi_k);
        for k=1:1:n_G
            for i=1:1:n_ne
                N_i = compute_N_e(Xi_k,n_G,i,k,n_ne);
                N(1,i,e,k)=N_i;
            end

            x_G(k,e) = N(:,:,e,k)*xe;
            b_k = c*exp(lambda*(x_G(k,e)));
            fe(:,e) = fe(:,e) + w_k(n_G,k)*Je(k)*N(:,:,e,k)'*b_k;
        end
        for i=1:1:n_ne
            f(Tn(e,i),1) = f(Tn(e,i),1) + fe(i,e);
        end
    end


    v_r = 1;
    v_f = setdiff(1:n,v_r);

    K_ff = K(v_f,v_f);
    K_fr = K(v_f,v_r);
    K_rf = K(v_r,v_f);
    K_rr = K(v_r,v_r);

    u(v_r,1) = 0;
    u(v_f,1) = K_ff\(f(v_f,1)-K_fr*u(v_r,1));
    f_r = K_rr*u(v_r,1) + K_rf*u(v_f,1)-f(v_r,1);



    sigma = zeros(n_G,n_el);
    sigma_G=zeros(n_G,n_el);
    for e=1:1:n_el
        u_e = u(Tn(e,:),1);
```

```
        for k=1:1:n_G
            sigma_G(k,e) = E*B(:,:,e,k)*u_e;
        end
    end

    sigma=interp1(x_G(:),sigma_G(:),x,'linear','extrap');

    sigma=sigma*10^(-6); %Mpa

    figure()
    plot(x,sigma,':o');
    hold on;
    plot(x_vector_exact,sigma_vector_exact)
    title("Stress along the 1D beam for " + n_el + " div. and " + n_G + " Gauss
    points")
    xlabel('x (m)')
    ylabel('\sigma (MPa)')
    legend('finite element solution','exact solution','Location','northeast')
    hold off;


u_exact_error=(x.*exp(4))/1800 - exp(2.*x)/3600 + 1/3600;

e_numerador = sqrt(sum((u_exact_error-u).^2));
e_denominador = sqrt(sum((u_exact_error).^2));
error(cont_n_el,n_G) = e_numerador/e_denominador;

end


end


figure()
loglog(h_vector,error(:,1),'-o')
hold on
loglog(h_vector,error(:,2),'-.+')
loglog(h_vector,error(:,3),'--x')
title("Accuracy plot")
xlabel('log h')
ylabel('log e')
legend('n Gauss = 1','n Gauss = 2','n Gauss = 3','Location','southeast')
```

8.2 Jacobian function code

```
function [Je, dN_dXie] = compute_Jacobian_e(n_G,n_ne,x,e,Tn,Xi_k)

for r=1:1:n_ne
    x_n(r) = x(Tn(e,r));
end
if n_ne == 2
    for jj=1:1:n_G
        for ii=1:1:n_ne
            if ii==1
                dN_dXie(ii,jj) = -1/2;
            else
                dN_dXie(ii,jj) = 1/2;
            end
        end
        Je(1,jj) = dN_dXie(1,jj)*x_n(1) + dN_dXie(2,jj)*x_n(2);
    end
else
```

```
    for jj=1:1:n_G
        xi = Xi_k(n_G,jj);
        for ii=1:1:n_ne
            if ii==1
                dN_dXie(ii,jj) = xi-1/2;
            elseif ii==2
                dN_dXie(ii,jj) = -2*xi;
            else
                dN_dXie(ii,jj) = xi+1/2;
            end
        end
        Je(1,jj) = dN_dXie(1,jj)*x_n(1) + dN_dXie(2,jj)*x_n(2) + dN_dXie(3,jj)*x_n
    (3);
    end
end

end
```

8.3 N function code

```
function N = compute_N_e(Xi_k,n_G,i,k,n_ne)

if n_ne == 2
    if i==1
        N = (1/2)*(1-Xi_k(n_G,k));
    elseif i==2
        N = (1/2)*(1+Xi_k(n_G,k));
    end
else
    if i==1
        N = (1/2)*Xi_k(n_G,k)*(Xi_k(n_G,k)-1);
    elseif i==2
        N = 1-(Xi_k(n_G,k))^2;
    elseif i==3
        N = (1/2)*Xi_k(n_G,k)*(Xi_k(n_G,k)+1);
    end
end

end
```

8.4 Exact solution function code

```
function [u_exact, du_exact] = compute_exact_solution(A,E,L,c,lambda)

syms x u(x)

b = c*exp(lambda*(x));
Du = diff(u,x);
u_exact = dsolve(diff(u,2)==-b/(E*A), u(0)==0, Du(L)==0);
du_exact = diff(u_exact,x);
end
```